Momentum Resolution

Florian Hellwig, Felix Huber, Mariano Fasano, Manuel Tuor

April 2025

1 Introduction

This project contains a simulation of particle trajectories with and without a magnetic field, as well as the reconstruction of those trajectories and the measurement of the particles' momentum using particle tracking detectors. In the following, the x-axis is vertical and the z-axis is horizontal.

In the first part, the situation is evaluated without any magnetic field. This means the particles, starting at some x-coordinate x_0, propagate toward positive z along a straight trajectory. The angle to the z-axis is expressed by s_0. During this trajectory, which is modeled by

$$x = x_0 + \tan(s_0) \cdot z = x_0 + \text{slope} \cdot z,$$

the particle crosses five detection planes that measure the x-position with cells that have a width of 500 micrometers (no resolution within the cell). The data from these detectors is then used to reconstruct the parameters x_0 and s_0, and thus the trajectory. This is first plotted for one particle before being repeated for 1000. Using the data from these thousand fits, the residuals and pulls are analyzed to determine the quality of the estimated parameters and uncertainties.

In the second part, the additional magnetic field introduces a new complexity. The five detection planes from the first part remain, but after those, a magnetic field alters the particle's path from a straight to a curved trajectory due to the Lorentz force. In the homogeneous magnetic field with strength B, the trajectory of a particle with charge q follows a circular path with radius of curvature

$$\rho = \frac{p_T}{q \cdot B},$$

where p_T is the transverse momentum. The bending angle of the particle can be estimated using

$$\theta \approx \frac{L}{\rho} = \frac{L \cdot q \cdot B}{p_T},$$

where L is the length of the magnetic field. Using the latter equation, we can approximate the transverse momentum using the reconstructed bending angle. This is illustrated by the following figure:

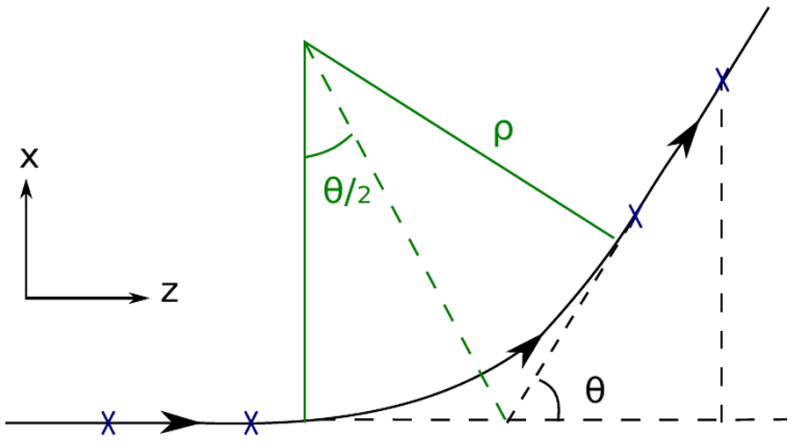


Figure 1: p_T can be reconstructed using the bending angle θ.

After the magnetic field, there are three additional detection planes of the same quality as the five preceding it. Since the magnetic field ends at the first of these three detectors, the particle once again propagates along a straight trajectory. Using the hit points from these detection planes, as well as those before the magnetic field, the straight segments of the particle's trajectory—and thus the bending angle, calculated as the difference between the angles of the two straight segments—can be reconstructed. Finally, as mentioned, we use the second equation above to obtain a reconstruction of the transverse momentum p_T.

Tracking detectors are used today at CERN, for example in the LHCb experiment, and have enormous importance in particle physics.

2 Experimental Setup

2.1 Tracking resolution

- **Detectors**: $n = 5$ detection planes that measure the x coordinate with a cell width of $500\,\mu$m. No resolution within the cells is provided. The detectors are spaced with $\Delta z = 2$ cm and are of infinite length.

- **Particles**: First, one example particle, then $n_{\text{trajectories}} = 1000$ particles starting at x_0, which is sampled from $N(0\,\text{mm}, 0.1\,\text{mm})$, propagating towards positive z with slope (i.e. angle) s_0 sampled from $N(0\,\text{rad}, 0.1\,\text{rad})$.

2.2 Momentum resolution

- **Detectors**: $n_{\text{beforeB}} = 5$ detection planes that measure the x coordinate with a cell width of $500\,\mu$m before and $n_{\text{afterB}} = 3$ detectors after the magnetic field. No resolution within the cells is provided. The detectors are spaced with $\Delta z = 2$ cm between each other and are of infinite length. The fifth detector is where the magnetic field begins, and the first one after (sixth in total) is where the magnetic field ends. This distance between the fifth and sixth detection plane is as long as the length of the magnetic field.

- **Magnetic field**: Homogeneous magnetic field of length $L = 10$ cm, located between the fifth and sixth detectors (zero outside). The magnetic field has field lines orthogonal to both the z and x axes. The magnetic field strength can vary with $B \in \{0.5, 1.0, 1.5, 2.0\}$ T.

- **Particles**: First, one example particle with momentum $p_T = 0.3\,\text{GeV}/c$, then $n_{\text{trajectories}} = 1000$ particles starting at x_0, which is sampled from $N(0\,\text{mm}, 0.1\,\text{mm})$, propagating towards positive z with slope (i.e. angle) s_0 sampled from $N(0\,\text{rad}, 0.1\,\text{rad})$. The particles can be have charge $+1$ and -1 and have transverse momentum $p_T \in \{0.1, 0.3, 1, 2, 5, 10, 20\}$ GeV/c.

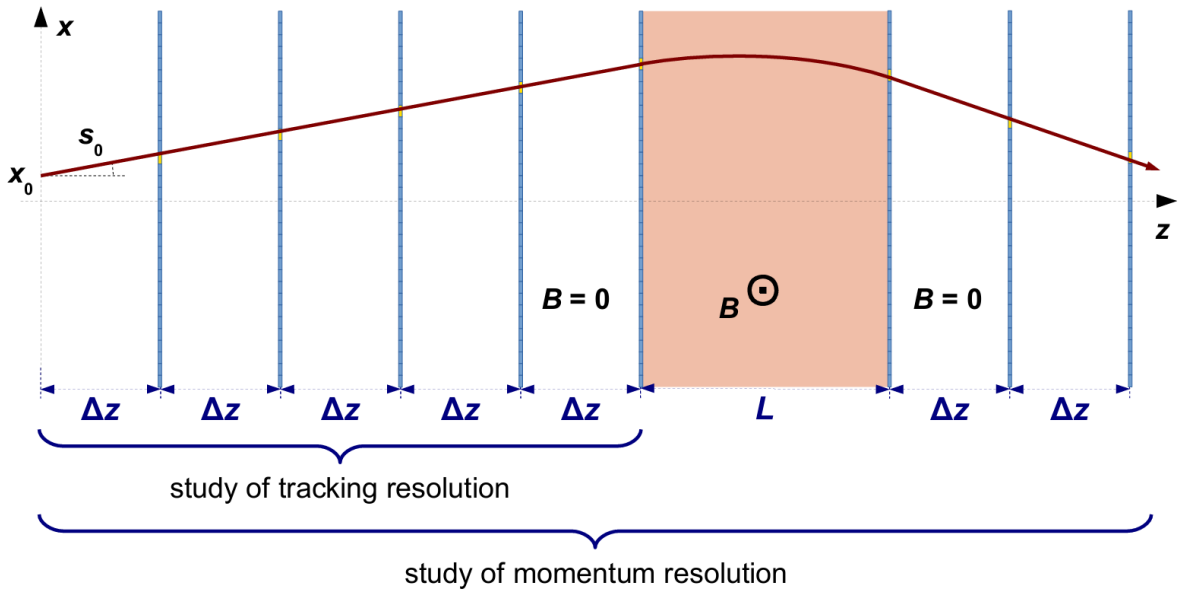


Figure 2: Setup of the experiment

3 Methodology

3.1 Tracking resolution

In the first part, only the detection planes before the magnetic fields are studied. Therefore, $n = 5$ planes are generated with $\Delta z = 2\,\text{cm}$ spacing between them. As a first warm-up exercise, only a single particle trajectory is simulated, starting at $z_0 = 0$ and $x_0 = x_0^{\text{true}}$, moving in the positive z-direction with "slope" $s_0 = s_0^{\text{true}}$. Note that s_0 is taken to be the angle, so the actual slope is calculated as $\tan(s_0)$. x_0^{true} is drawn from a Gaussian distribution with mean 0 and standard deviation 1 mm, and s_0^{true} from a Gaussian distribution with mean 0 and standard deviation 0.1 rad.

Then, the hit positions on the detection planes are determined—that is, which cells are hit. The detection planes are made up of cells with a width of $500\,\mu$m. The center of the hit cell is taken as the measured point, and the uncertainty from a uniform distribution, $\sigma_x = \text{cell width}/\sqrt{12}$, is used in the fitting process.

This process of fitting was improved multiple times: first, an attempt was made to fit x_0^{reco} and s_0^{reco} (the angle) using `scipy.optimize.curve_fit`. However, this significantly underestimated the uncertainties, since `curve_fit` was actually fitting the function `x0 + np.tan(s0) * z_position`. To avoid this nonlinearity, the fit was reformulated to use the actual slope instead of the angle. Since this still resulted in a significant (though smaller) underestimation of the uncertainties, the issue was resolved by using a custom function, `manual_least_squares`, which implements weighted linear regression.

Using the reconstructed parameters x_0^{reco} and the "slope" s_0^{reco} (for the angle), the original and reconstructed trajectories are plotted to illustrate the overall situation.

Afterwards, the simulation is extended to 1000 trajectories. The impact parameter x_0^{reco} and reconstructed slope s_0^{reco}, along with their uncertainties, are again calculated for each trajectory, as in the warm-up exercise, in order to evaluate the detector resolution in x_0 and s_0. Histograms of the residuals and pulls of s_0^{reco} and x_0^{reco} are created. The mean and standard deviation are computed, and for each histogram of the pulls, the ideal Gaussian models with $\mu = 0$, $\sigma = 1$ are added for comparison. This approach is taken to avoid bias and incorrect estimation of the uncertainty, and was crucial in the process of finding the best fitting method.

Besides the already mentioned weighted linear regression (which is actually also used in the second part), this part also uses error propagation, the uncertainty estimates of a gaussian distribution, and other nice implementations. The code is fully commented and can be found in the appendix. Also, the code using `curve_fit` is outcommented and can be easily activated.

3.2 Momentum Resolution

For this second part, the trajectory and detection planes are the same as in the tracking resolution section. The particle then passes through the magnetic field. As it is charged, it experiences a Lorentz force, causing it to follow a circular path within the magnetic field. After exiting the field, it once again moves as a free particle along a straight line and hits the pixels in the three detection planes positioned behind the magnetic field.

As before, to begin, a single particle trajectory is simulated for a particle with transverse momentum $p_T = 0.3\,\text{GeV}/c$. The parameters x_0^{true} and s_0^{true} are again drawn from the same distributions as previously. The particle's charge is randomly assigned to be either positive or negative, i.e., $q = +1$ or $q = -1$. Now we model the trajectory of this particle (it is also possible to set the number of particles to a larger number, as seen below). Using the previously mentioned equation,

$$\rho = \frac{p_T}{q \cdot B},$$

we calculate the curvature radius, which we then use—combined with the initial slope, charge, and endpoint of the first straight trajectory (and a lot of geometry and trigonometry)—to determine the center of the circular path onto which the particle is pushed by the Lorentz force. These center coordinates are calculated by the function `circular_path_center_coordinates()`.

This information is then used by the function `circular_path_coefficients()`, which calculates the points of the circular trajectory in the magnetic field. Again, geometry and trigonometry are used. Finding the end of the arc was quite a challenge. Our original approach was the following: The angle `theta_end`, which can be calculated as the difference between the bending angle `theta` and the initial angle of the arc, `theta_start`, should represent the final angle of the arc. While `theta_start` can be calculated using trigonometry and the differences between the circle center and start of the arc, finding the exact `theta` is not that easy. We used the approximation

$$\theta \approx \frac{L \cdot q \cdot B}{p_T}.$$

However, when simulating multiple trajectories, this resulted in a too-early cut-off of the arcs deviating the most from the midline. The solution was to use `scipy.optimize.root_scalar()` to find the exact (or rather a very good approximation of) `theta_end` individually for each arc. Using this approach, we could visually simulate the trajectories for an arbitrary number of particles (see below for 100 and 1000), albeit at the cost of a longer running time.

Using the normal of the vector between the circle center and the last point of the arc, the slope and starting angle of the straight trajectory after the field are obtained (done by the function `starting_angle_after_B()`)—and the last point of the arc is, of course, the new (x_0, z_0) for the straight trajectory after the magnetic field.

The hit positions are recorded for all detection planes, and the same pixel uncertainty as in the first part is applied. A straight line is fitted to the trajectory both before and after the magnetic field, and the bending angle is determined by the difference between the angle after the field and the one before it (or at the beginning). Solving the approximation of the angle for the momentum, we obtain an approximate equality with which we can reconstruct the transverse momentum p_T^{reco}. Initially, this was done straightforwardly, and the uncertainty was calculated using error propagation. However, because the fits of the angle before and after the magnetic field are correlated, the uncertainty is often underestimated.

To address this, we added a Monte Carlo sampling of each of the two angles, using the mean `angle_before_B_field` and standard deviation `angle_before_B_field_err` (analogously for the angle after the magnetic field). The mean of the differences became the reconstructed transverse momentum p_T^{reco}, and the standard deviation served as the standard error of the reconstruction (function `compute_pT_reco`). This improved the means of the pulls slightly (but significantly), but often overestimated the uncertainty, especially for large transverse momenta (which makes sense, as a large momentum leads to big standard errors of the angles which makes the differences of the angles more (and apparently too) distributed). Therefore, we included both functions in the code, with the original one chosen by default. This can be changed via the parameter `pt_calculation_way`.

This procedure is then repeated for 1000 particles. In addition to plotting all 1000 trajectories (visible in the notebook), histograms of the residuals and pulls of p_T^{reco} are created. The mean and standard deviation are computed for each histogram, as well as their standard errors, and compared to the expected Gaussian distributions with $\mu = 0$ and $\sigma = 1$. Finally, the exercise is repeated for various true transverse momenta with fixed magnetic field strength $B = 0.5$ T:

$$p_T^{\text{true}} = 0.1,\ 1,\ 2,\ 5,\ 10,\ 20\,\text{GeV}/c,$$

and for fixed $p_T^{\text{true}} = 0.3\,\text{GeV}/c$ with varying magnetic field strengths B:

$$B = 1.0,\ 1.5,\ 2.0\,\text{T}.$$

The resolution is evaluated for these different parameters.

4 Results

4.1 Tracking resolution

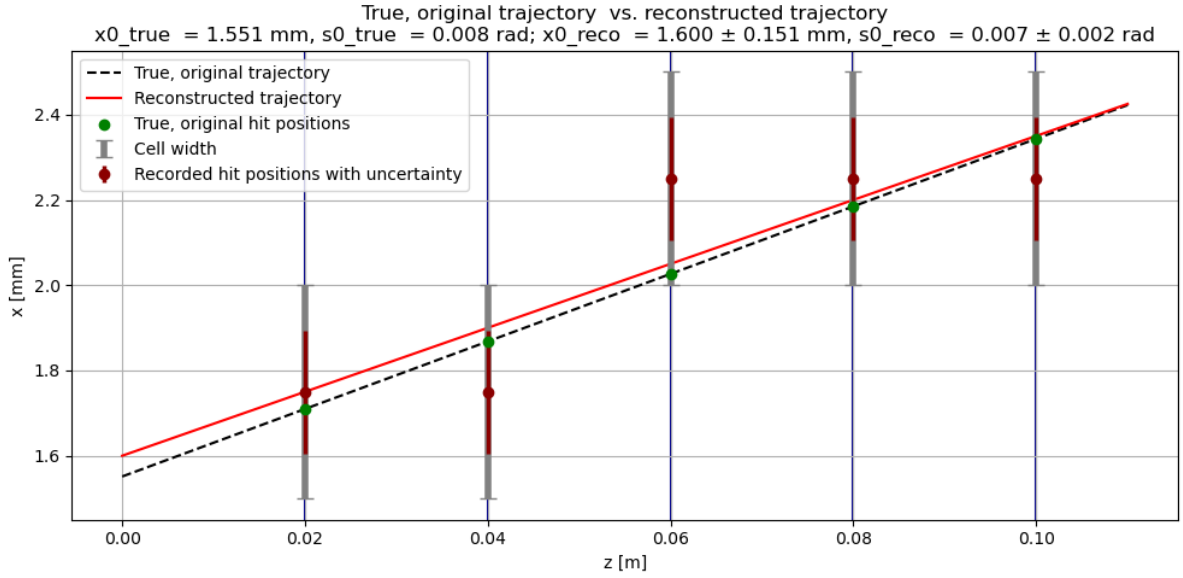


Figure 3: Warm-up: Figure of a single trajectory (in black and dashed) and its reconstruction (countinuous line in red). Further elements, estimates, and uncertainties are also indicated.

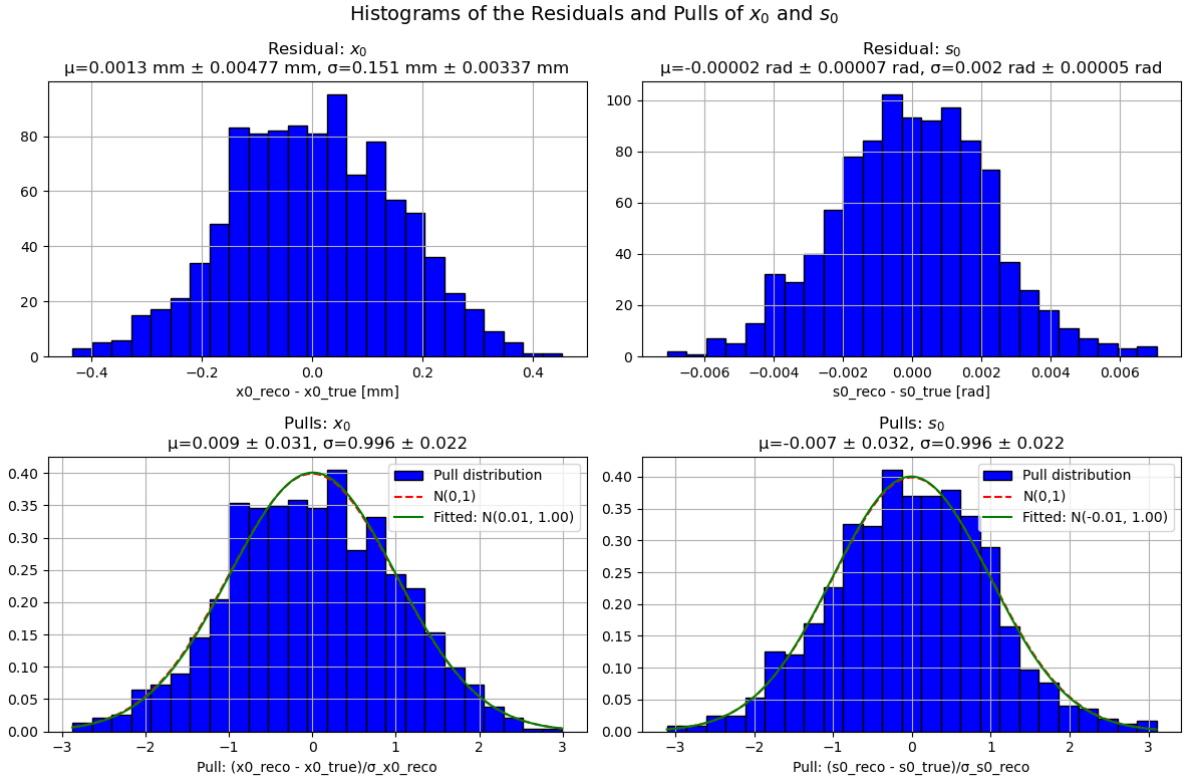


Figure 4: Histograms of the residuals and the pulls of x_0 and s_0. The estimates of the parameters of the distributions are indicated and the pulls are compared to a standard normal distribution.

There are no significant under- or overestimations of the parameters. All results are reproducible.

The first part could be completed very successfully, with a very high quality indicated by the pulls.

4.2 Momentum resolution

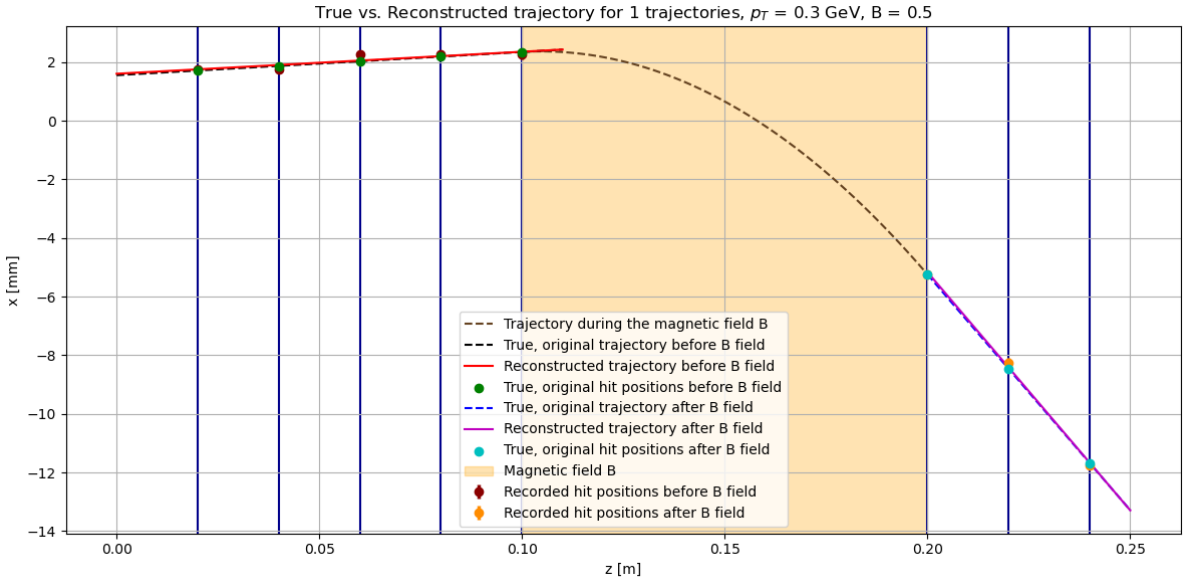


Figure 5: Warm-up: Visual illustration of a single trajectory (in dashed) and the reconstruction of the straight parts of its trajectories (continuous line in red and violet)

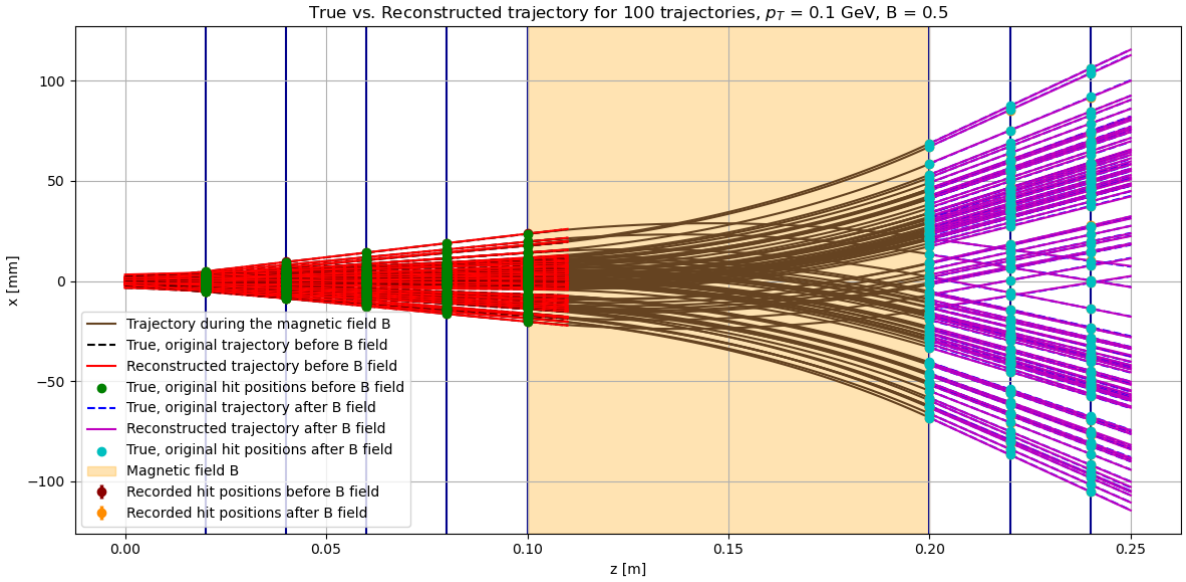


Figure 6: Extension of warm-up: Visual illustration of a hundred trajectories and the reconstruction of the straight parts of its trajectories (in red and violet). Note that the true (not reconstructed) trajectories in the magnetic field are drawn as continuous lines to make it easy to follow an individual trajectory.
If wanted, this can be repeated for an arbitrary number of trajectories using the code.

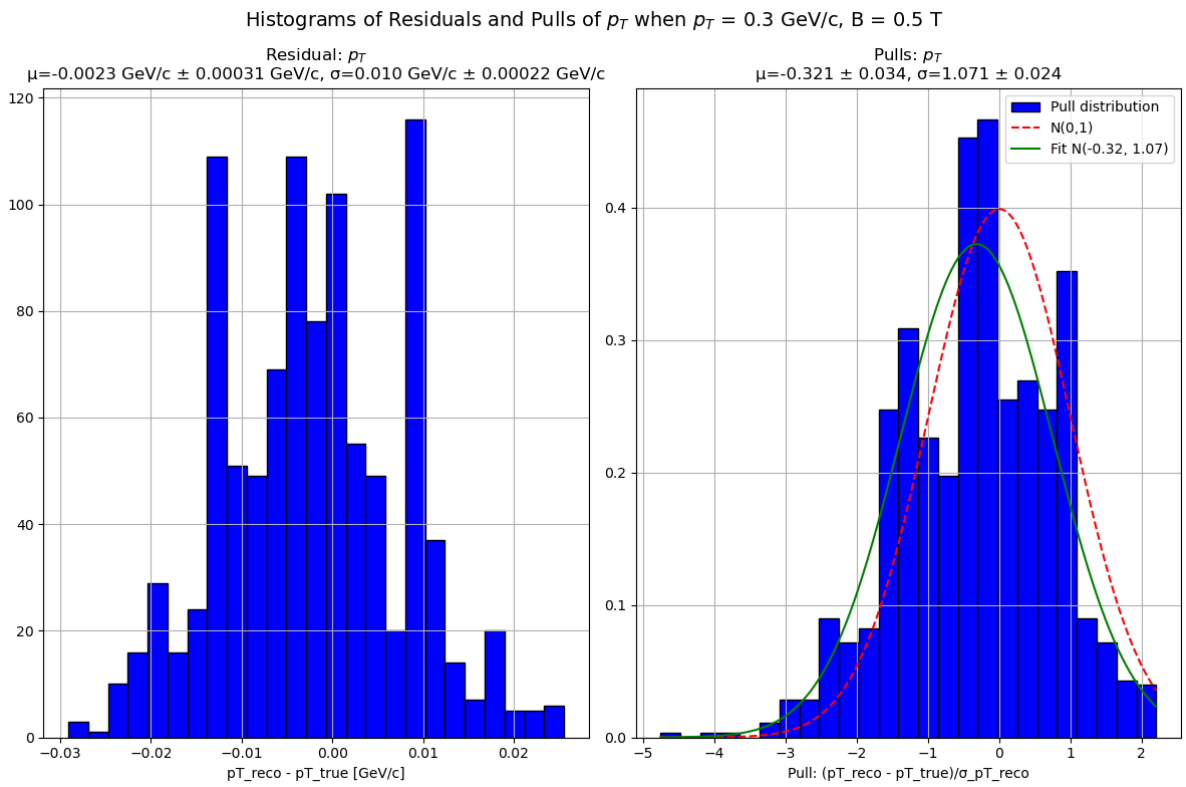
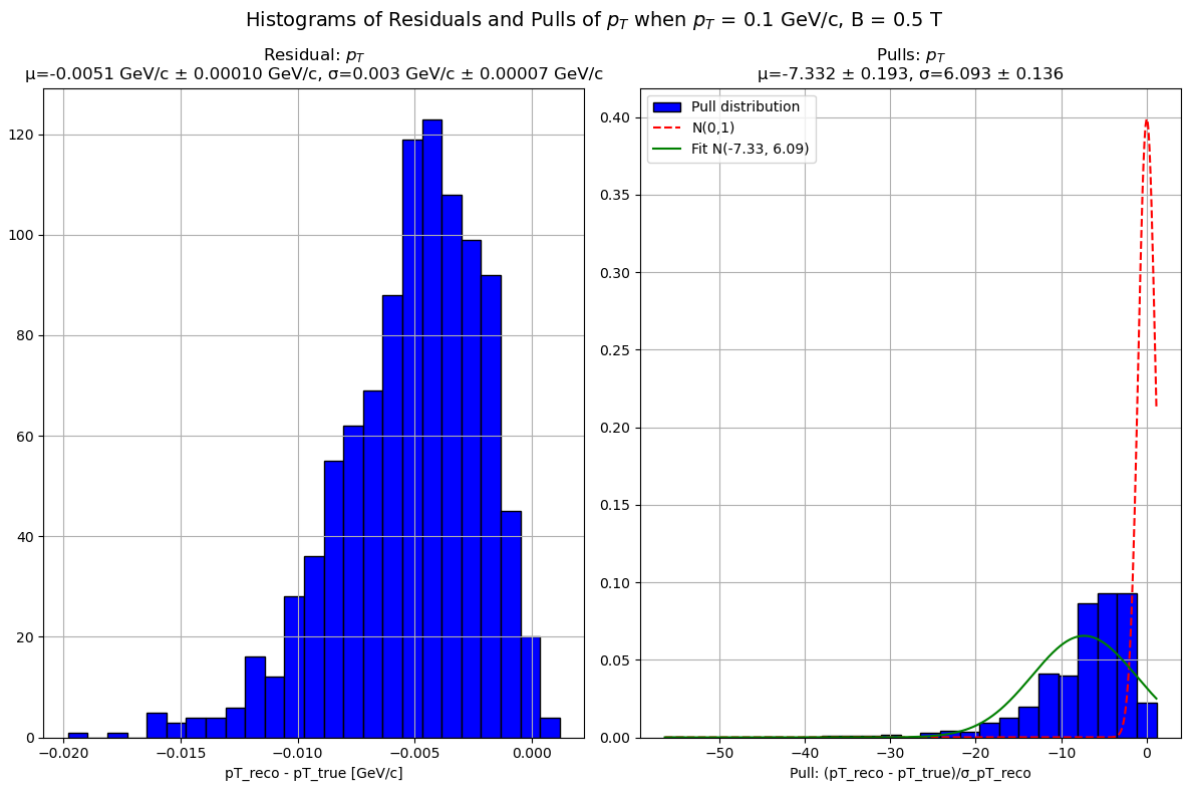
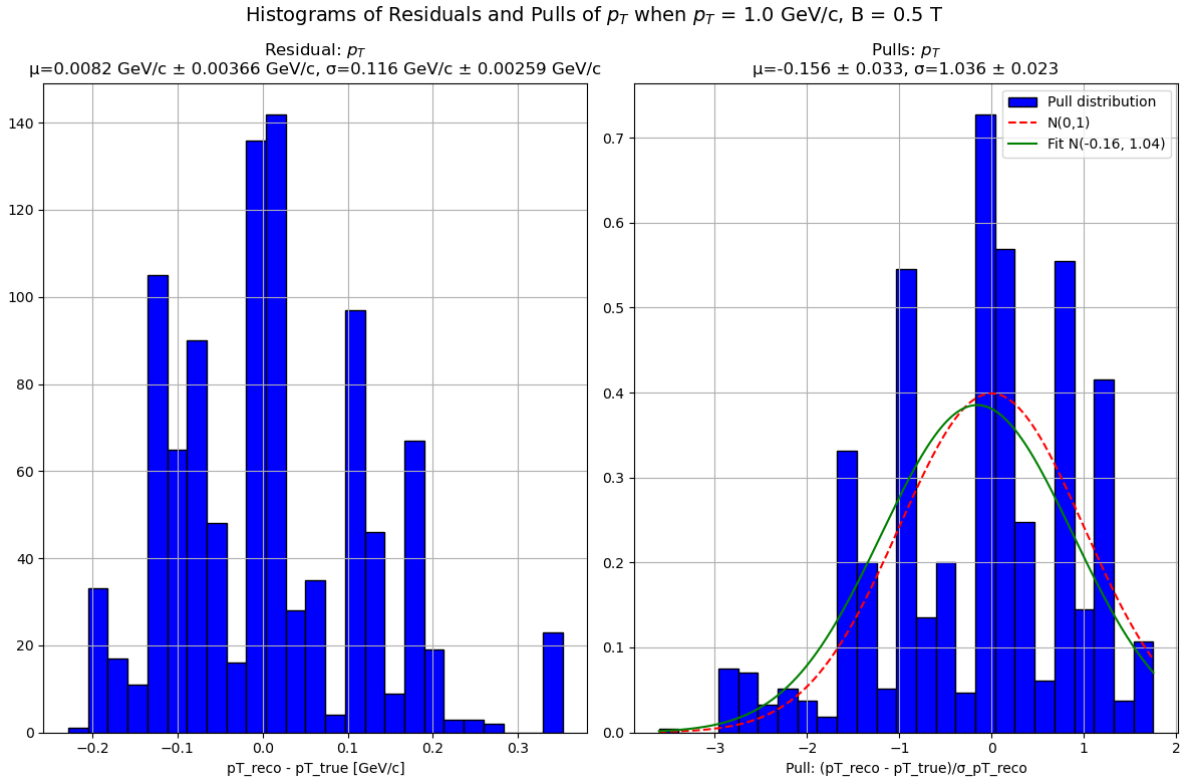


Figure 7: Histograms of the residuals and the pulls of the reconstruction of p_T when $p_T^{\text{true}} = 0.3$ GeV and $B = 0.5$ T.

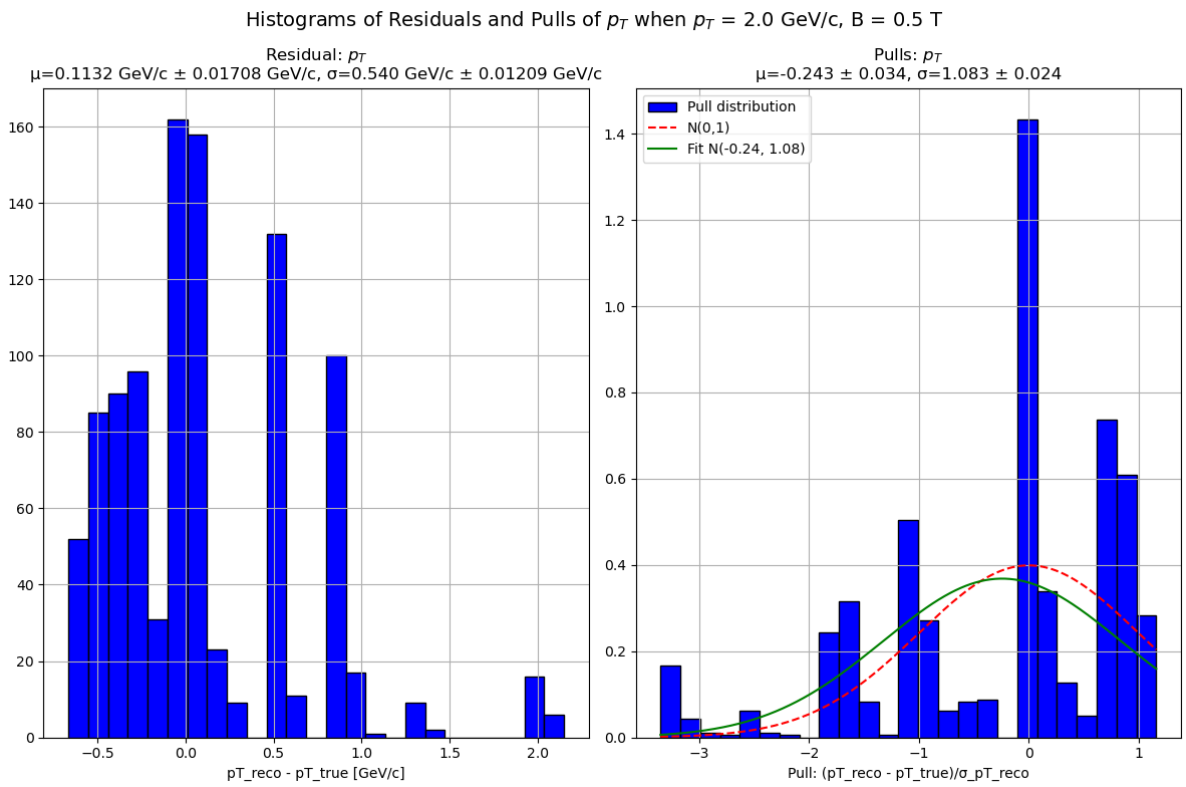


(a) $p_T^{\text{true}} = 0.1$ GeV/c

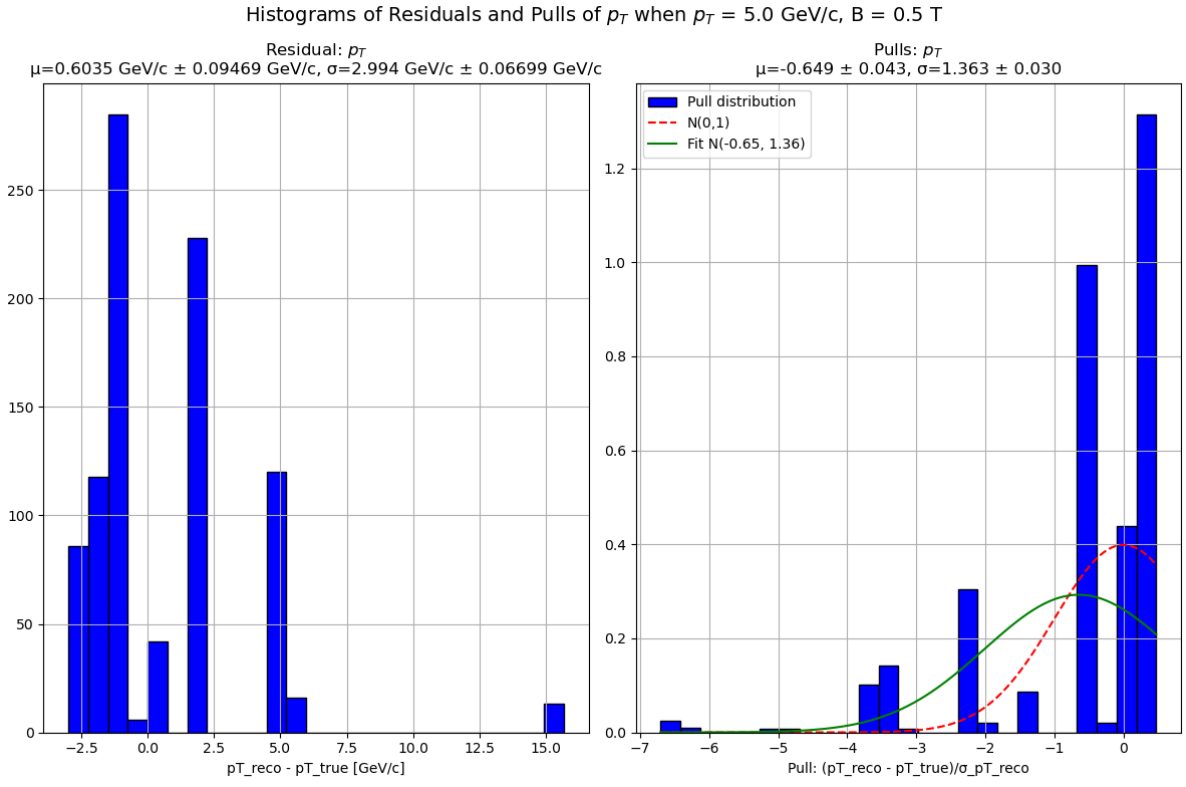


(b) $p_T^{\text{true}} = 1.0$ GeV/c

Figure 8: Histograms of the residuals and the pulls of the reconstruction of p_T when p_T^{true} varies and $B = 0.5$ T is fixed. (Plot 1/3)

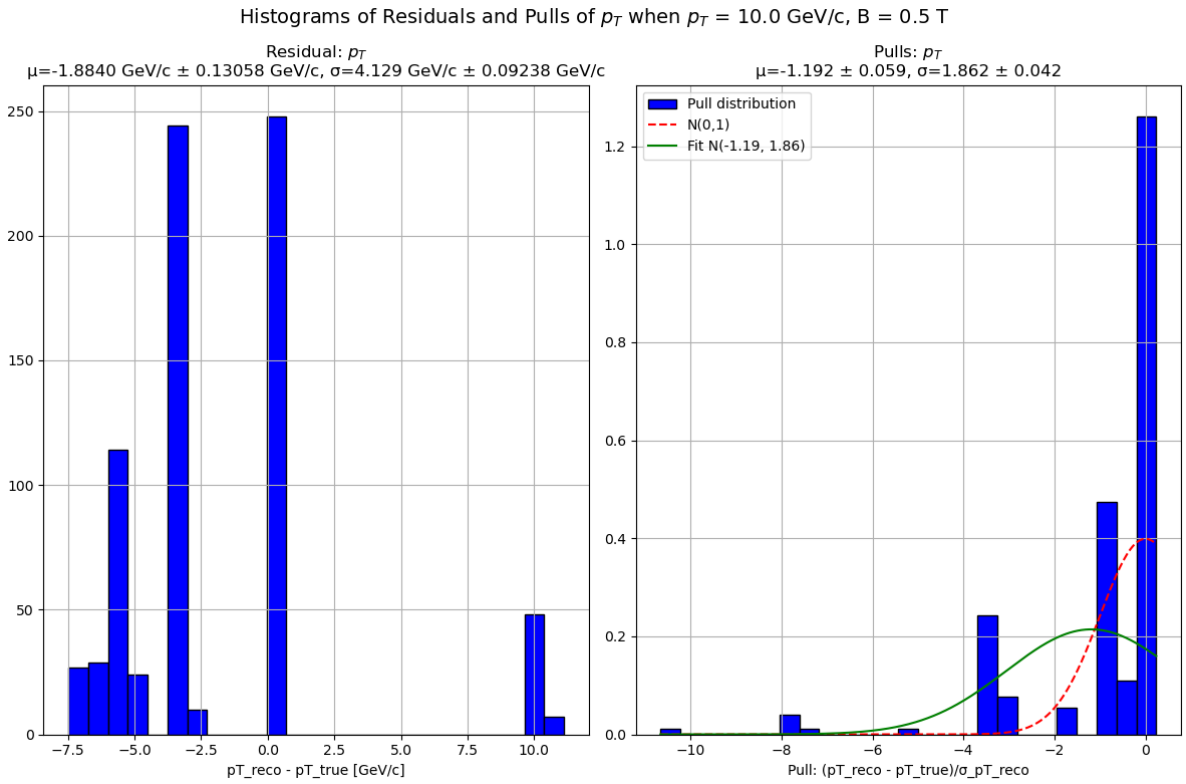


(a) $p_T^{\text{true}} = 2.0$ GeV/c

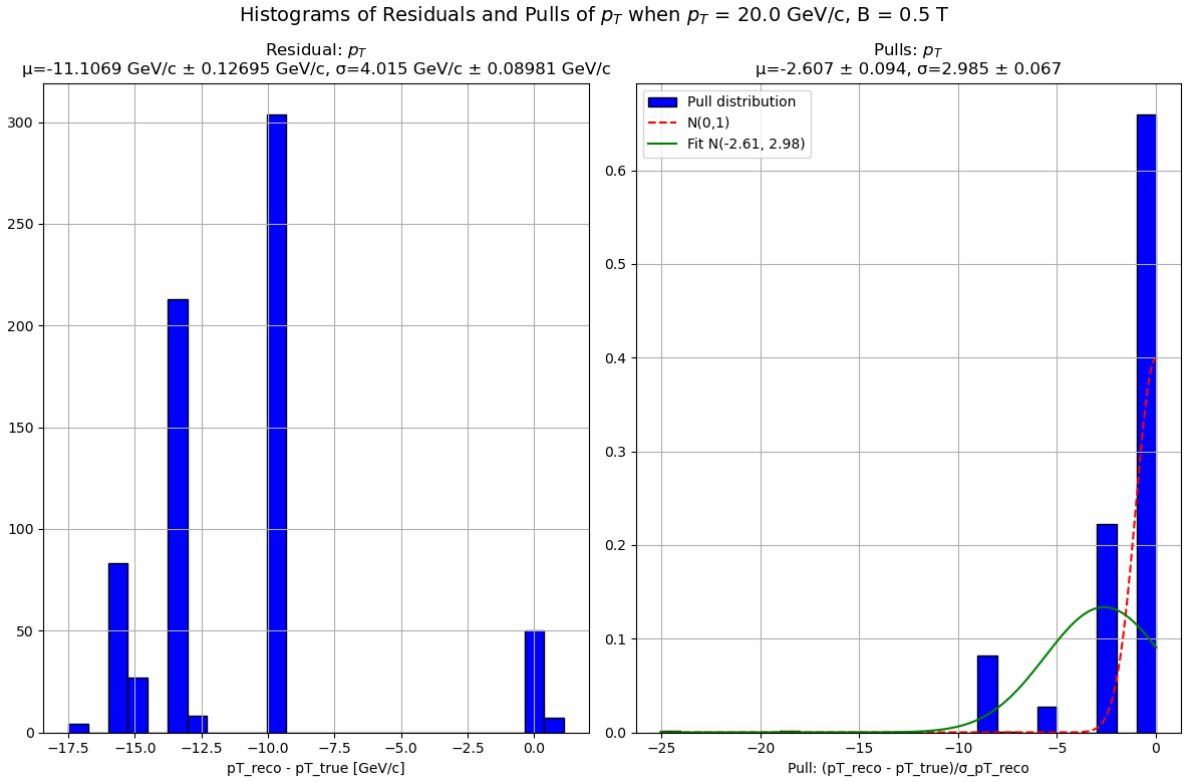


(b) $p_T^{\text{true}} = 5.0$ GeV/c

Figure 9: Histograms of the residuals and the pulls of the reconstruction of p_T when p_T^{true} varies and $B = 0.5$ T is fixed. (Plot 2/3)

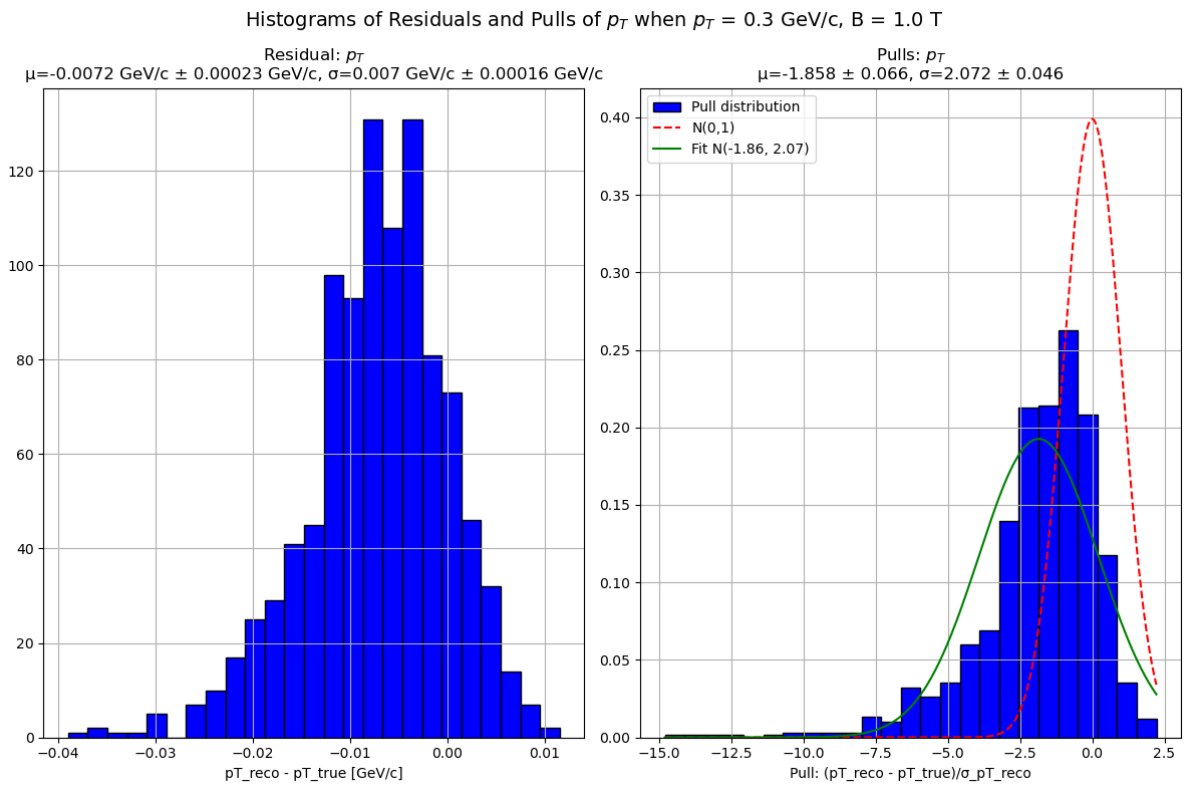


(a) $p_T^{\text{true}} = 10.0$ GeV/c

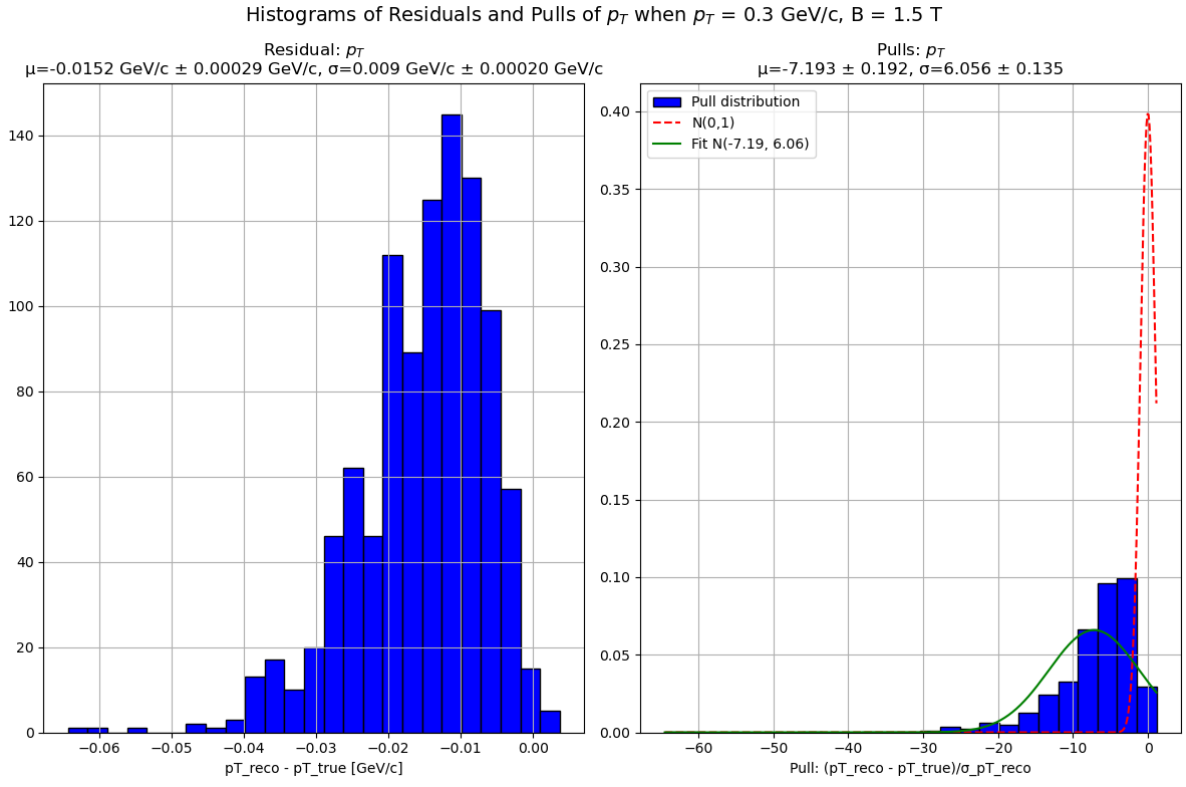


(b) $p_T^{\text{true}} = 20.0$ GeV/c

Figure 10: Histograms of the residuals and the pulls of the reconstruction of p_T when p_T^{true} varies and $B = 0.5$ T is fixed. (Plot 3/3)

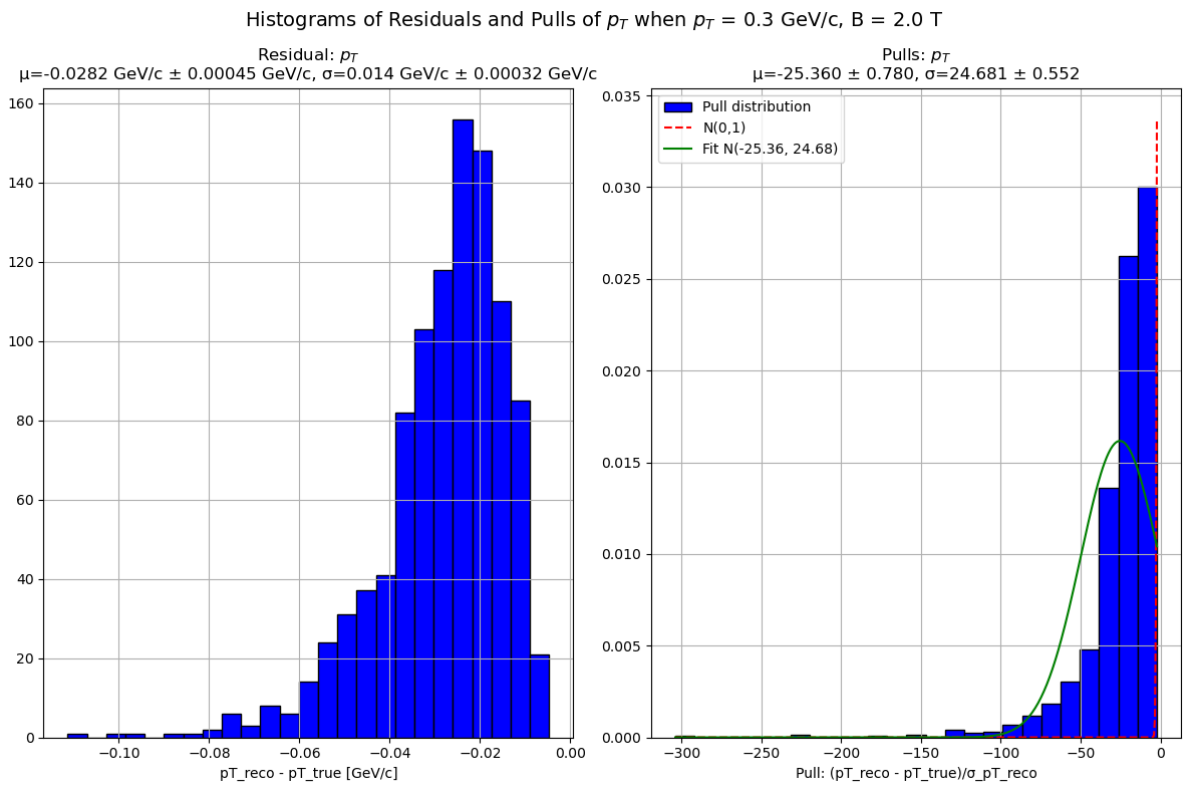


(a) $B = 1.0$ T



(b) $B = 1.5$ T

Figure 11: Histograms of the residuals and the pulls of the reconstruction of p_T when $p_T^{\text{true}} = 0.3$ GeV fixed and B varies. (Plot 1/2)



(a) $B = 2.0$ T

Figure 12: Histograms of the residuals and the pulls of the reconstruction of p_T when $p_T^{\text{true}} = 0.3$ GeV fixed and B varies. (Plot 2/2)

We can observe the following:

1. There appears to be a bias in the reconstruction, as all the pull histograms show a negative mean. Moreover, with larger values of B or more extreme values of p_T^{true}, this bias becomes stronger. We conclude that the reconstructed bending angle `theta` (`angle_after_B_field` − `angle_before_B_field`) is overestimated. This results from the fact that there are only three detection planes after the magnetic field, which are relatively close together. Since the slope is inversely proportional to the z-distance between two hits, even a small error in the x-direction can lead to large errors in the measured angle. Generally, the slopes are overestimated due to an asymmetry introduced by the cell widths: although it is equally probable that the slope is reconstructed too small or too large, the deviations are larger when the slope is overestimated. This leads to an overestimation of the angle after the magnet, and thus to an underestimation of the transverse momentum p_T^{reco}.

2. Often—especially for large values of p_T^{true} and B—the uncertainties are underestimated (standard deviation of the pulls greater than 1). This occurs because of a correlation between the two angles (before and after the magnetic field). This correlation, along with some further inaccuracies introduced due to the accumulation of errors is also strengthened by the step-by-step calculation approach (first the center of the circle, then the points of the arc, then the normal vector of the vector going from the center to the last point of the arc) we chose. These uncertainties are propagated to calculate the uncertainty in the reconstructed p_T (error propagation). However, due to the correlation, the propagated uncertainty is underestimated.

3. As p_T^{true} increases, the histogram increasingly deviates from a Gaussian shape and becomes segmented into distinct parts—i.e., the randomness increases. This is caused by the

discretisation of the y-axis (i.e., the detection planes). At higher momentum and speed, this effect becomes more pronounced. Using smaller cell widths and a smaller or more variable momentum can help mitigate this issue.

4. Although only partly visible in the plots, a stronger magnetic field B is expected to improve the resolution of the transverse momentum. This is because a higher magnetic field leads to a smaller radius of curvature ($\rho = \frac{p_T}{q \cdot B}$), meaning the particle bends more. This increased deflection makes it easier to distinguish between fast and slow particles, improving momentum resolution.

5. Similarly, a smaller true transverse momentum p_T^{true} should result in better resolution of the reconstructed transverse momentum p_T^{reco}, as it leads to a smaller radius of curvature ($\rho = \frac{p_T}{q \cdot B}$), causing more pronounced bending and, as before, better resolution. This trend is visible more or less in the plots—at least more clearly than in the case of varying B.

5 Error Calculation and weighted linear regression

5.1 Uncertainty estimation of the measured hit points

The uncertainty of the measured hit points was estimated using the uncertainty from a uniform distribution and set to:

$$\sigma(\text{hit position}) = \frac{\text{cell width}}{\sqrt{12}},$$

where σ(hit position) is the standard error (and therefore a measure of the uncertainty) of the hit position.

Note that this estimate does not account for multiple scattering, which could significantly influence the uncertainty of the hit position in a real experiment.

5.2 Uncertainty estimation of Gaussian distribution

For the estimation of the uncertainties of the parameters of the residuals and pulls, the classic uncertainty formulas of Gaussian distributions were used:

$$\sigma(\mu) = \frac{S_x}{\sqrt{n}},\ \sigma(S_x) = \frac{S_x}{\sqrt{2n-2}},$$

where μ is the mean of the distribution, S_x is the standard deviation of the distribution, and $\sigma(p)$ is the standard error (and therefore a measure of the uncertainty) of the parameter p.

5.3 Uncertainty estimation using error propagation

To avoid nonlinearities when fitting and to improve the estimations of the uncertainties from `curve_fit` and later to be able to use the weighted linear regression, we used the equation `x0 + slope * z` for the fitting procedure (note that the slope here refers to the actual slope and not an angle). Both methods return an estimate of the slope and its standard error. The slope (better: $\text{slope}^{\text{reco}}$) is then converted back to the angle using `s0_reco = np.arctan(slope_reco)`. To calculate the standard error of s_0^{reco}, we use error propagation. In general, this can be expressed as:

$$y = f(x) \implies \sigma(y) = \left| \frac{d}{dx} f(x) \right| \sigma(x).$$

When applied to $s_0^{\text{reco}} = \arctan(\text{slope}^{\text{reco}})$, we obtain:

$$\sigma(s_0^{\text{reco}}) = \frac{\sigma(\text{slope}^{\text{reco}})}{1 + (\text{slope}^{\text{reco}})^2}.$$

This was used in both parts of the project.

Additionally, we also used error propagation to get the standard error of p_T^{reco}:

$$p_t^{\text{reco}} = f(s_{0\text{before Magnetic field}}, s_{0\text{after Magnetic field}}),$$
$$\text{with } f(s_{0\text{bMf}}, s_{0\text{aMf}}) = \frac{q \cdot B \cdot L}{s_{0\text{aMf}} - s_{0\text{bMf}}}$$

$$\begin{aligned}\sigma(p_T^{\text{reco}}) &= \sqrt{\left(\sigma(s_{0\text{bMf}}) \cdot \frac{\partial f}{\partial s_{0\text{bMf}}}\right)^2 + \left(\sigma(s_{0\text{aMf}}) \cdot \frac{\partial f}{\partial s_{0\text{aMf}}}\right)^2} \\ &= \sqrt{\left(\sigma(s_{0\text{bMf}}) \cdot \frac{q \cdot B \cdot L}{(s_{0\text{bMf}} - s_{0\text{aMf}})^2}\right)^2 + \left(\sigma(s_{0\text{aMf}}) \cdot \frac{-q \cdot B \cdot L}{(s_{0\text{bMf}} - s_{0\text{aMf}})^2}\right)^2}\end{aligned}$$

5.4 Calculation of pulls

The pulls of a parameter p are useful for checking the quality of the reconstruction p^{reco} and its uncertainty estimate $\sigma(p^{\text{reco}})$. A pull of the parameter p is calculated as follows:

$$\text{pull}(p) = \frac{\text{reconstructed quantity} - \text{generated quantity}}{\text{uncertainty on reconstructed quantity}} = \frac{p^{\text{reco}} - p^{\text{true}}}{\sigma(p^{\text{reco}})}.$$

Using multiple pulls, we can study their distribution. If the reconstruction is unbiased and the uncertainty is estimated correctly, this distribution should follow a standard normal distribution, i.e., $\mu = 0$, $\sigma = 1$. Deviations in the mean μ imply a bias in the reconstruction, while deviations in the standard deviation σ indicate incorrect uncertainty estimates.

5.5 Weighted linear regression

To improve the estimation of `scipy.otimize.curve_fit` which still underestimated the uncertainty after implementing a linear function to fit, we decided to implement a weighted linear regression. As this was not in the course and new to us, we quickly want to explain the theory of the method implemented in the function `manual_least_squares` in the code.

We want to fit a straight line to the datapoints $(z_i, x_i), i = 1, ..., n$. From above we know that we assume the same uncertainty $\sigma_i = \frac{\text{cell width}}{\sqrt{12}}$ for every point. From the theory, we know that the weights w_i have the property $w_i = \frac{1}{\sigma_i^2}$.

We now want to minimise the weighted sum of squares:

$$\chi^2 = \sum_{i=1}^{n} w_i(x_i - (x_0 + mz_i))^2 = \sum_{i=1}^{n} \frac{(x_i - (x_0 + mz_i))^2}{\sigma^2},$$

where x_0 is the x-intercept (i.e. the x-value of the trajectory at $z = 0$), m is the slope, and $\sigma = \sigma_i$ for some i (this works as all σ_i are the same).

We now want to minimise this term, meaning that we derive it both by x_0 and by m and set the resulting terms $= 0$. We get:

$$\frac{d\chi^2}{dx_0} = -2\sum_{i=1}^{n} w_i(x_i - (x_0 + mz_i) = 0$$

and

$$\frac{d\chi^2}{dm} = -2\sum_{i=1}^{n} w_i z_i(x_i - (x_0 + mz_i) = 0.$$

Rearraging this, we receive the two normal equations:

$$\sum_{i=1}^{n} w_i x_i = x_0 \sum_{i=1}^{n} w_i + m \sum_{i=1}^{n} w_i z_i$$

and

$$\sum_{i=1}^{n} w_i z_i x_i = x_0 \sum_{i=1}^{n} w_i z_i + m \sum_{i=1}^{n} w_i z_i^2.$$

We now set

$$S_w = \sum_i w_i,\ S_{wz} = \sum_i w_i z_i,\ S_{wx} = \sum_i w_i x_i,$$

$$S_{wz^2} = \sum_i w_i z_i^2, : S_{wzx} = \sum_i w_i z_i x_i.$$

Then the two equations can be written as

$$\begin{pmatrix} S_w & S_{wz} \\ S_{wz} & S_{wz^2} \end{pmatrix} \begin{pmatrix} x_0 \\ m \end{pmatrix} = \begin{pmatrix} S_{wx} \\ S_{wzx} \end{pmatrix}$$

We now solve this equation by multiplying with the inverse matrix from the left and (using the adjoint to compute the inverse), we get:

$$\begin{pmatrix} x_0 \\ m \end{pmatrix} = \frac{1}{S_w S_{wz^2} - S_{wz}^2} \begin{pmatrix} S_{wz^2} & -S_{wz} \\ -S_{wz} & S_w \end{pmatrix} \begin{pmatrix} S_{wx} \\ S_{wzx} \end{pmatrix}.$$

Therefore,

$$x_0^{\text{reco}} = \frac{S_{wz^2} S_{wx} - S_{wz} S_{wzx}}{S_w S_{wz^2} - S_{wz}^2}$$

and

$$m^{\text{reco}} = \frac{-S_{wx} S_{wz} + S_w S_{wzx}}{S_w S_{wz^2} - S_{wz}^2}.$$

The covariance matrix is given by:

$$\begin{pmatrix} S_w & S_{wz} \\ S_{wz} & S_{wz^2} \end{pmatrix}^{-1} = \frac{1}{S_w S_{wz^2} - S_{wz}^2} \begin{pmatrix} S_{wz^2} & -S_{wz} \\ -S_{wz} & S_w \end{pmatrix}.$$

We therefore use

$$\sigma(x_0^{\text{reco}}) = \sqrt{\frac{S_{wz^2}}{S_w S_{wz^2} - S_{wz}^2}},\ \sigma(m^{\text{reco}}) = \sqrt{\frac{S_w}{S_w S_{wz^2} - S_{wz}^2}}$$

to get hold of the uncertainties.

This is implemented exactly like this by the function `manual_least_squares`. The code is commented well, so it should be easy follow.

6 Trigonometry to Calculate the Circular Path

To provide insight into our construction of the circular path in the magnetic field, we offer an explanation of each step.

`circular_path_center_coordinates()`: We calculate the center of the circle on which the circular pathway lies. Given are the x-coordinate and the z-coordinate of the start of the magnetic field (`x_start, z_start`) as well as the curvature radius (`radius`), calculated as $\frac{p_T^{\text{true}}}{B}$. We distinguish four cases, which are the possible combinations of the sign of the slope and the

charge. Here, only the case of a positive slope and positive charge is explained; all other cases can be (and were) calculated analogously:

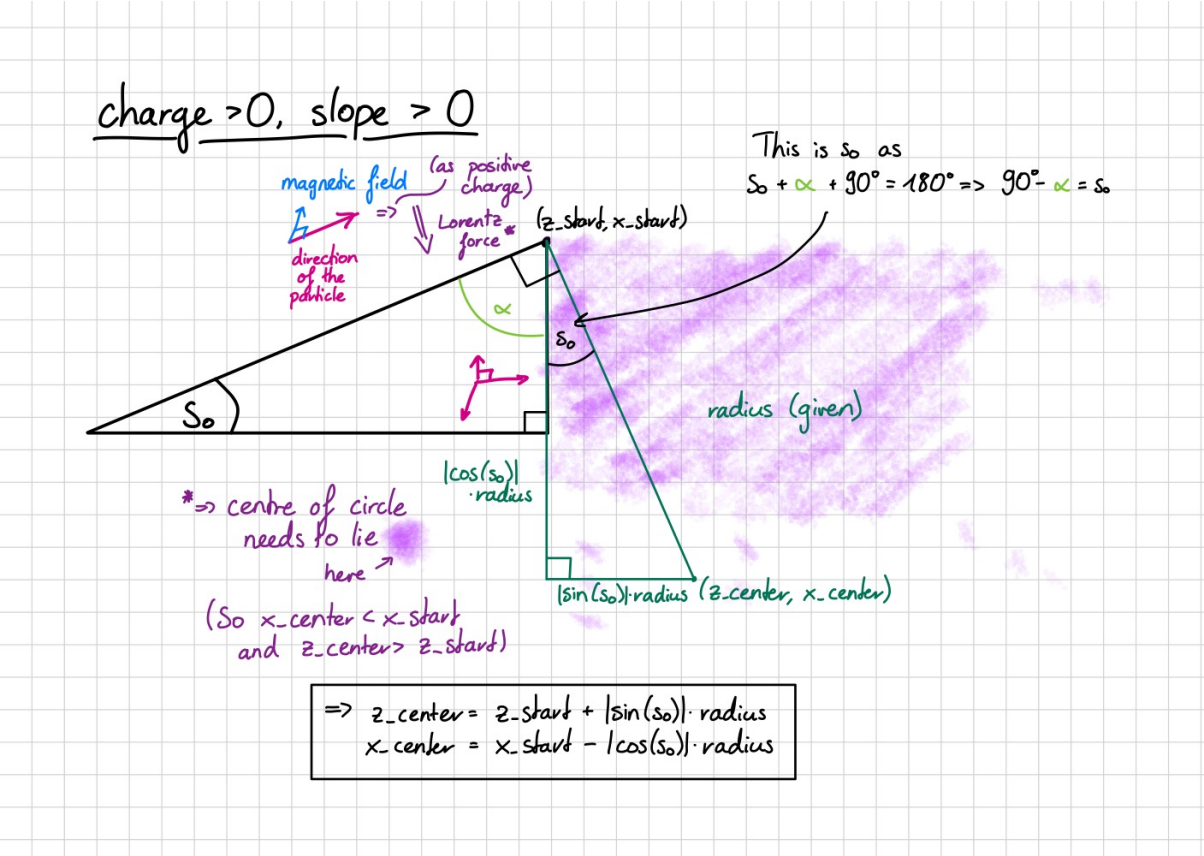


Figure 13: Explanation of how to calculate the coordinates of the centre of the circle. Note that the centre must lie on the normal to the incoming trajectory; this is why there is a right angle at (`z_start`, `x_start`).

`circular_path_coordinates()`: This function calculates the points lying on the arc. It first computes the initial angle of the arc (`theta_start`), then the final angle (`theta_end`) by finding the angle whose z-coordinate lies on (or very close to) the detection plane that ends the magnetic field. All intermediate angles are then used with standard trigonometry to calculate the arc coordinates. Note that all four situations from above are handled using the same code. This works because we allow for negative sine and cosine values, as well as negative components of `np.arctan2()`. In the following, the simplest case is illustrated:

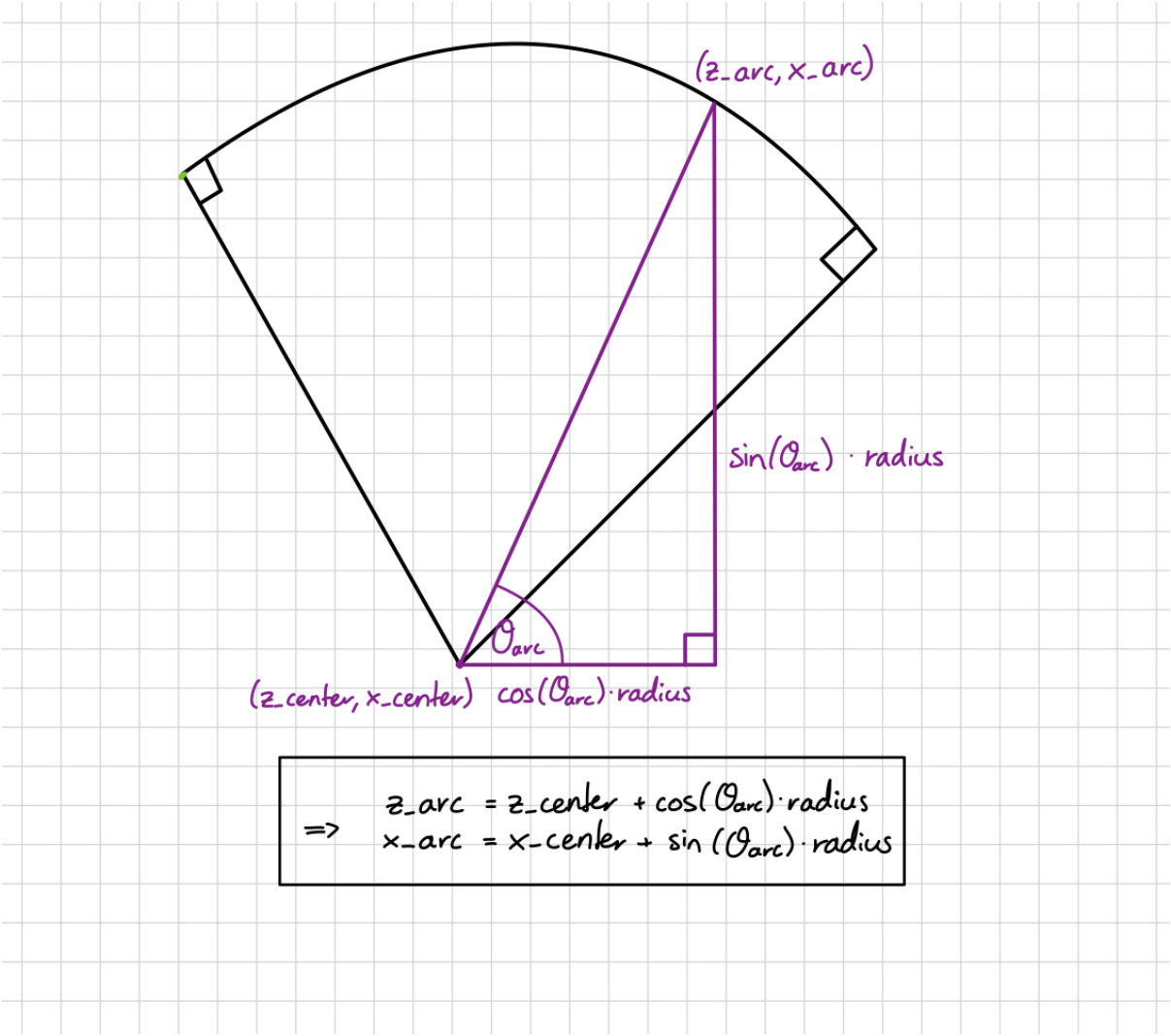


Figure 14: Explanation of how the arc coordinates were calculated.

`starting_angle_after_B()`: This function calculates the angle of the slope after the magnetic field. It does so by taking the arctangent of the normal vector of the vector going from the centre of the circle to the last point of the arc (which lies on or very close to the first detection plane after the magnetic field).

Using these three functions, we can reconstruct the circular path that the particle takes.

7 Conclusion, Outlook, and Potential Improvements

First, it is important to emphasize that it is not possible to directly compare the experiment conducted in this project to the complexity of the experiments at CERN. Significant simplifications were made to make the project feasible. For example, it was completely neglected that, in reality, the magnetic field is not zero at the boundaries. Additionally, in a real detector, multiple scattering effects play a role, but these were not considered in our analysis. Finally, the entire experiment was highly simplified, as we did not work with a real detector or real data, but instead used a simplified simulation intended to approximate a real experiment. It would be very interesting to analyze actual experimental data and to take into account more of the aforementioned factors.

While the first part of our project proceeded smoothly after resolving the issues caused by the

use of `curve_fit`—with the pulls following a standard normal distribution—the second part did not go that smoothly. There are noticeable distortions in the residuals and pulls, with the momentum and uncertainty of p_T^{reco} often being underestimated. While some of these issues can be attributed to the factors discussed earlier, improvements in this area would be desirable. We attempted to address this by using the new function for computing p_T^{reco} (`compute_pT_reco_mc()`) and making numerous other changes (which took dozens of hours), but we were not successful to the extent we had hoped. It may be worth questioning whether, with the mentioned step-by-step calculation approach we chose, perfect results are even achievable, or if a complete rewrite of the code would be necessary.

It is unfortunate that the influence and improvement of the momentum resolution, especially with varying B, is not clearly visible in our plots. Nevertheless, the physical principles behind momentum reconstruction and the role of the magnetic field strength, as well as the original momentum, were highly intriguing to us and led to many discussions—some of which extended beyond the scope of the project.

8 Appendix

In the following Git repository, all the code can be found: GitHub Repository. The final notebook is `Momentum_Resolution_full_project_extended.ipynb`. The display of the simulations with 1000 trajectories are outcommented (because it takes quite long to run), but can be easily activated.

Note: For the correction of language mistakes in this report (and only for that), AI was used.